Technical Report — Liquid Transfer v6 Safety Diagnostic

Friday, October 23, 2026
Author: Yema Quinn
Team: Natkeeran, Neil
Model: Model 3 (MuJoCo), PPO, known container positions

1. What I Did Today

Definitions used throughout this report:

- **Fluid delivery** — liquid physically moves from the beaker to the dish. This is what the original v4 reward measures.
- **Task success — fluid is delivered and the pipette never penetrates the beaker wall and no contact force exceeds 5 N. This is what the task actually requires.**
- **Penetration event** — the pipette tip crosses the beaker wall at an altitude below the rim ($z < 0.12$ m).

Setup and infrastructure

Step	Task	Outcome
1	Restored missing franka_emika_panda/ assets/ folder	Assets downloaded via DownGit
2	Resolved Python environment (Intel Mac: torch 2.2.2, MuJoCo 3.10.0, rsl_rl 2.3.3)	Working base env
3	Wrote liquid_transfer_m3_v6. py — v4/v6 selectable reward, geometric penetration detection, contact-force logging, safe-transit shaping	Ready

Environment validation (does the code do what it claims?)

Step	Task	Metric	Outcome
4	Sanity probe (-- mode probe)	IK reach error	0.0 cm, both envs, both containers

5	Scripted controller under v6, warmup on (default)	Task success	16/16 (100%), 0 penetration events, mean 208 steps
9	Scripted controller under v6, warmup off (--warmup_frac 0.0)	Task success	16/16 (100%), 0 penetration events, mean 208 steps

Scripted results at steps 5 and 9 are identical. This is the fairness check: v6 does not punish correct behavior. The gate fires only on trajectories that would have caused penetration, and the scripted controller never penetrates.

Diagnostic runs (what does the v4-trained policy actually do?)

Step	Task	Metric	Outcome
6	Diagnostic A: v4 checkpoint in v4 environment	Fluid-delivery rate	87.4% (132/151), mean 177 steps
7	Diagnostic B: v4 checkpoint in v6 environment, warmup on	Fluid delivery / penetration logged	0% fluid delivery; penetration logged but not terminating (warmup softens the gate; success fails on integrity check, not termination)
8	Diagnostic C: v4 checkpoint in v6 environment, warmup off (--warmup_frac 0.0)	**Task success / penetration terminations**	**0.2% task success (1/435); 434 penetration terminations**

Steps 6–8 form the causal chain. Under v4, the policy delivers fluid 87.4% of the time. Under v6, the same policy triggers penetration on the first horizontal step out of the beaker in 434 of 435 episodes, because the exploit it learned under v4 required travelling through the beaker wall. **The v4 result is a fluid-delivery rate, not a task-success rate. The v4 policy's task success rate is 0.2%.**

Evidence captured

Step	Task	Outcome

10	Screen recordings	Video of v4 policy exploiting under v6 (warmup off), and scripted controller executing correctly under v6

2. The Core Result

Metric	v4	v6 (warmup off)
Episodes	151	435
Fluid delivery	132 (87.4%)	1 (0.2%)
Task success (fluid + no penetration + no force)	not measured	1 (0.2%)
Integrity violations (penetration terminations)	n/a	434
Timeouts	19	0
Overfill	0	0
Mean episode length	177 steps	65 steps
Fluid delivery, dish near nominal (0–5 cm)	81.8%	0.0%
Fluid delivery, dish far (20+ cm)	92.0%	1.2%

What the v4 policy actually does: it dips the pipette to z ≈ 0.10 m, aspirates, and slides horizontally to the dish at the same altitude. The beaker rim is at z = 0.12 m, so the tip travels through the beaker wall on exit. The pipette geom has no collision (contype=0, conaffinity=0), so physics permits it. **The v4 reward only checks fluid in the dish, so the exploit is rewarded identically to the intended behaviour.** The 87.4% figure is **not a success** rate. It is a **fluid-delivery rate** in which the delivery mechanism destroys the container.

What the same policy does under v6: the geometric penetration check fires on the first horizontal step out of the beaker, so 434 of 435 episodes terminate before fluid reaches the dish. Mean episode length collapses from 177 to 65 steps — the policy is caught almost immediately. The 1 remaining "success" is likely a

rare episode where the policy happened to lift high enough to clear the rim; we did not isolate it.

Task success rate for the v4 policy, measured correctly: 0.2%.

3. The Scripted Control (Fairness Check)

The scripted controller lifts to z = 0.25, aspirates at z = 0.08, lifts back to z = 0.25, moves, dispenses at z = 0.06, lifts away. It never travels below the rim while outside a container.

Test	Reward	Warmup	Result
Scripted under v6	v6	on	16/16 success, mean 208 steps
Scripted under v6	v6	off	16/16 success, mean 208 steps

v6 does not punish correct behavior. The gate fires only on trajectories that would have caused penetration. Scripted performance is identical with warmup on or off. This rules out the possibility that v6 is simply over-penalizing.

4. v6 Reward Design (What Changed)

v6 = v4 + four additions:

Term	Formula	Value	Purpose
contact_force	−0.5 × max(0, F_n − 5N) per step	dense	Discourage any beaker/dish contact
penetration_dense	−5 per penetration event	dense	Immediate signal on wall cross
penetration_term	−50, one-shot	terminal	Hard stop on exploit
safe_transit	+5/step, capped at 50/episode	dense	Teach the lift behavior

Penetration detection: geometric. Track d_beaker (XY distance from tip to beaker centre) and z_tip each step. A penetration event fires when:

- Previous step: d < R_beaker AND z < z_rim (inside, low)
- Current step: d ≥ R_beaker AND z < z_rim (crossed the wall while still below rim)

This is a pure geometric check on the tip site. It does not rely on collision geometry, which is why it catches the exploit even though the pipette has no collision mesh.

Success redefinition:
text
```
fluid_delivered = |dish_ml − target_ml| < 0.3
```

(v4's definition of 'success');
```
 task_success = fluid_delivered AND penetration_count == 0 AND
max_contact_force < 5 N
```
(v6)

Fluid delivery is necessary but not sufficient. Clean execution is also required.

Warmup: --warmup_frac 0.20 (default) softens the termination constraint for the first 20% of training steps so exploration can find the lift behavior. During warmup, penetration is penalized (−5 per event) but does not terminate. After warmup, it terminates at −50. In play mode, I override with --warmup_frac 0.0 to test the strict policy.

6. Environment Notes (for reproducibility)

If either of you wants to run this on your machine:

- Intel Mac constraint: torch is capped at 2.2.2 (no further x86_64 macOS wheels). rsl_rl 3.x needs torch 2.5+, so it can't run here. I installed rsl_rl 2.3.3 with --no-deps, but its OnPolicyRunner has an API mismatch with the env. Play mode was solved by loading policy.pt directly with torch.load and running the actor MLP by hand (_load_policy_from_policy_checkpoint in the file). Training on the Intel Mac would require bypassing rsl_rl entirely.
- MuJoCo vieIr: on macOS, mjpython is required (not python). This is documented in MuJoCo's vieIr.py.
- MuJoCo headless: MUJOCO_GL=cgl (not egl, which is Linux-only).
- Assets: the franka_emika_panda/assets/ folder must be inside model_3_mujoco_ppo_known_positions/, not a sibling. Natkeeran — this might explain why the assets loaded for you without issue; check your repo layout.
- Folder colon: School:Education in the iCloud path breaks uv venv (PATH separator). Renamed to School - Education. Not a project issue, but worth knowing if either of you hits an install error with a colon in a path.

7. Interpretation and Implications

7.1 For our project

The v4 architecture, environment, and training pipeline are sound. The failure is specification, not implementation. This is a known class of RL failure — reward hacking or specification gaming — and it's the same class that produces famous examples like the boat racing game agent that circles instead of finishing (to farm points) or the robot that learns to flip the object rather than grasp it.

My contribution is that I caught it, diagnosed it, and closed it — in a domain (self-driving labs) where reward specification is safety-critical. In a real SDL, a pipette passing through the wall of a sterile container would contaminate the culture, and a robot that "succeeds" by doing so is not a robot that can be deployed.

7.2 For the report

The two videos + two [Play] outputs give us a clean before/after narrative:

1. v4 policy achieves 87.4% by exploiting a reward gap.
2. v6 catches the exploit on 434/435 episodes.
3. v6 does not punish correct behavior (scripted 16/16).
4. Therefore v6 is a valid safety layer, not an over-correction.

7.3 For the showcase

Dr. Maurice will want to see this. It's not a demo of "look how lll the robot works." It's a demo of "look how I found a flaw in the reward function, proved it, and fixed it." That's a research contribution — the second kind of result is more interesting to an academic audience than a clean success curve, because clean success curves are everywhere and diagnosed failures are rare.

8. Next Steps

Immediate (this weekend)

1. Training under v6. Path A: patch rsl_rl 2.3.3 to accept our env's get_observations return type (10-line shim). Path B: write a self-contained PPO that runs on torch 2.2.2 (180 lines, no external dependency). Path B is more work tonight but more robust and more defensible in the report.
2. ~~Update README with v4 vs v6 experiment commands and the two [Play] outputs as evidence.~~
3. Screenshots for the report slide: crop the [Play] summaries side by side.

Before the showcase

1. Train v6 to convergence (or document why it doesn't converge — sparse-reward exploration is the same wall).
2. Add a physical fix as a second comparison: put a collision sphere on the

pipette tip in panda_pipette.xml, and run v4 policy against it. This proves the exploit is a physics-vs-reward choice, not a rendering artifact.
3. Draft the 5-slide presentation: task, two failure modes, v6 fix, scripted control, conclusion.

Open questions for the team

1. Natkeeran — does your environment's panda_pipette.xml have the same contype=0, conaffinity=0 on the pipette geom? If yes, the exploit likely reproduces on your Isaac Lab setup too (once I remove any implicit collision). Worth checking before you defend the 96%.
2. Neil — the environmental-disturbance curriculum stage you proposed (moving the beaker/dish) is directly testable under v6. If you want to run that experiment once v6 training stabilizes, it slots in as a curriculum phase in _sample_offsets.
3. All — do I want to pivot the framing from "our robot delivers fluid" to "our robot delivers fluid safely, and I can prove the difference"? That's the stronger claim, and the report already supports it.

9. Short Version

Under the original reward, the policy delivers fluid 87% of the time — but it's doing so by sliding the pipette through the beaker wall. Under the patched reward, fluid delivery collapses to 0.2% because 434 out of 435 episodes terminate on first wall penetration. Under the patched reward, success collapses to 0.2% because 434 out of 435 episodes terminate on first wall penetration. I checked the patch doesn't punish correct behavior: the scripted controller still gets 16/16. So we know: the model learned the exploit, not the task, and we have a fix that's fair and catches it. I have video of both. The next step is training under the patched reward and seeing whether it can learn the safe version of the task.

10. One-line summary

Natkeeran's model (v4) delivers fluid 87% of the time. It just does it by smashing the beaker. That's not success — that's a reward function that didn't know the difference. My diagnostic exposes the gap and patches it.